

Introduction to FastAPI

Building High-Performance Backend APIs with Python



What is FastAPI?

- A modern, fast web framework for Python.
- The bridge between your database and your user interface.

Why Choose FastAPI?



- **High Performance:** On par with Node.js and Go.



- **Rapid Development:** Increases coding speed by 200% to 300%.



- **Fewer Bugs:** Reduces developer-induced errors.



- **Intuitive:** Great for students and beginners to learn.



How an API Works

The Customer: Your Streamlit Frontend (requests data).

The Waiter: Your FastAPI Backend (carries the request).

The Kitchen: Your Database / OpenAI (prepares the data).



Your First Endpoint

Creating an API endpoint is as simple as writing a Python function.

We use decorators (like `@app.get()`) to tell FastAPI which URL triggers the function.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to the  
AI Chatbot API\!"}
```



The Four Core Operations

POST: Create new data
(e.g., Start a new chat).

GET: Read existing data
(e.g., Load chat history).

PUT: Update existing data.

DELETE: Remove data
(e.g., Delete a chat session).



Dynamic URLs with Path Parameters

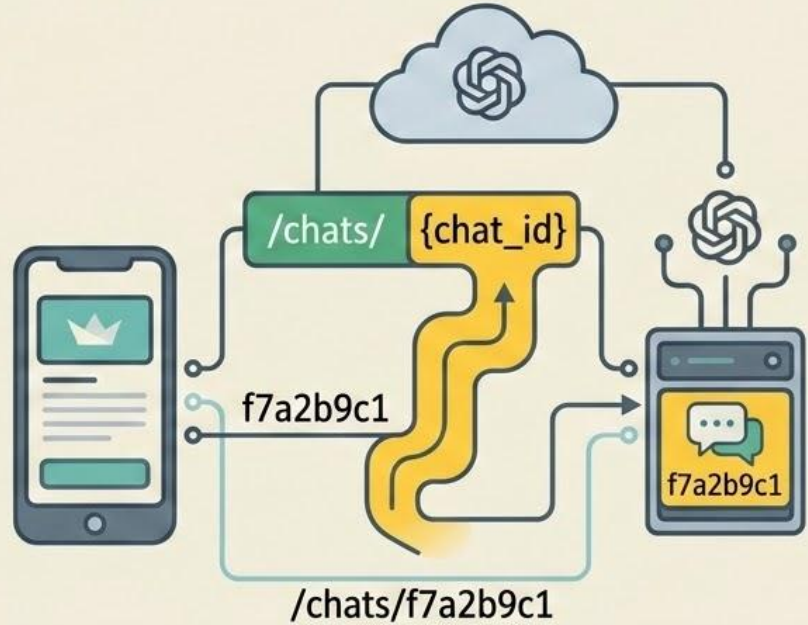
You can extract variables directly from the URL.

Useful for identifying specific items, like a unique **chat_id**.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/chats/{chat_id}")
def get_chat(chat_id: str):
    # The URL /chats/f7a2b9c1 makes
    return {"current_chat": chat_id}
```



Validating Data with Pydantic

FastAPI uses Pydantic to ensure incoming data is correct.

If a user sends a number instead of text, FastAPI automatically rejects it.

```
from pydantic import BaseModel

class MessageRequest(BaseModel):
    message: str

@app.post("/send")
def send_message(req: MessageRequest):
    return {"received_text": req.message}
```

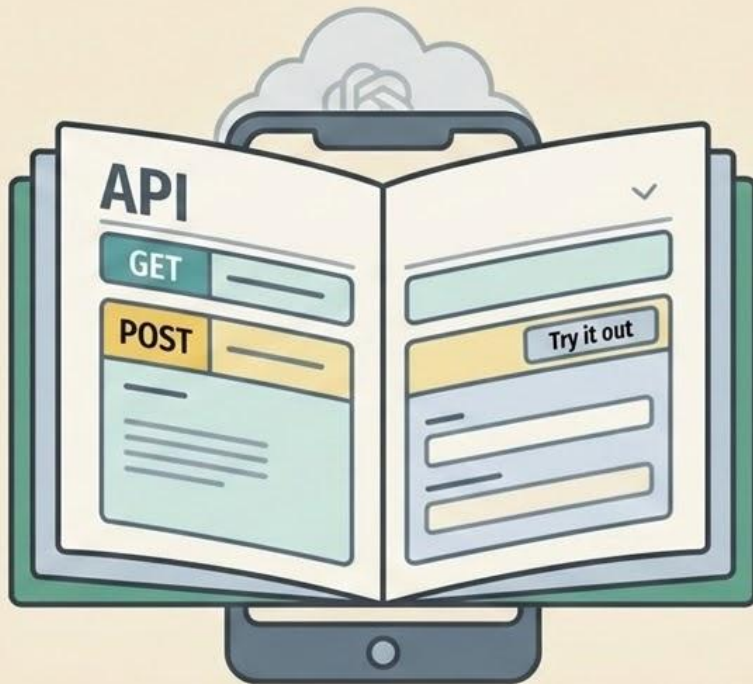


Free Interactive Docs (Swagger UI)

FastAPI automatically generates a testing webpage for your API.

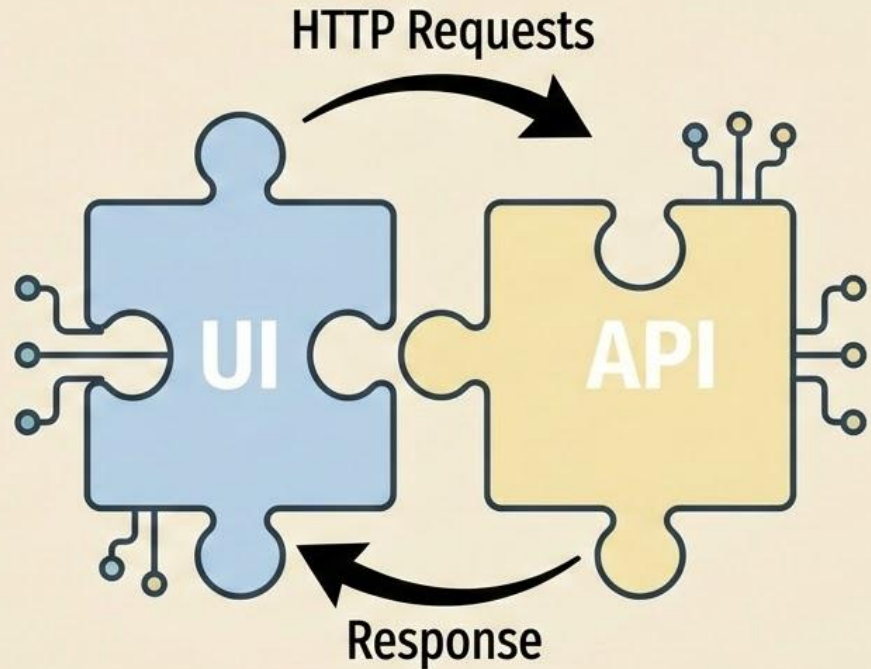
Just visit `/docs` in your browser.

- Allows you to test endpoints without building a frontend first.



Full-Stack Integration

- The backend (FastAPI) runs on port 8000.
- The frontend (Streamlit) runs on port 8501.
- Streamlit uses the Python requests library to talk to FastAPI over HTTP.



Ready to Build!

- FastAPI: Handles logic, database, and AI communication.
- Streamlit: Handles buttons, inputs, and display.
- Separating these concerns makes you a better software engineer.

